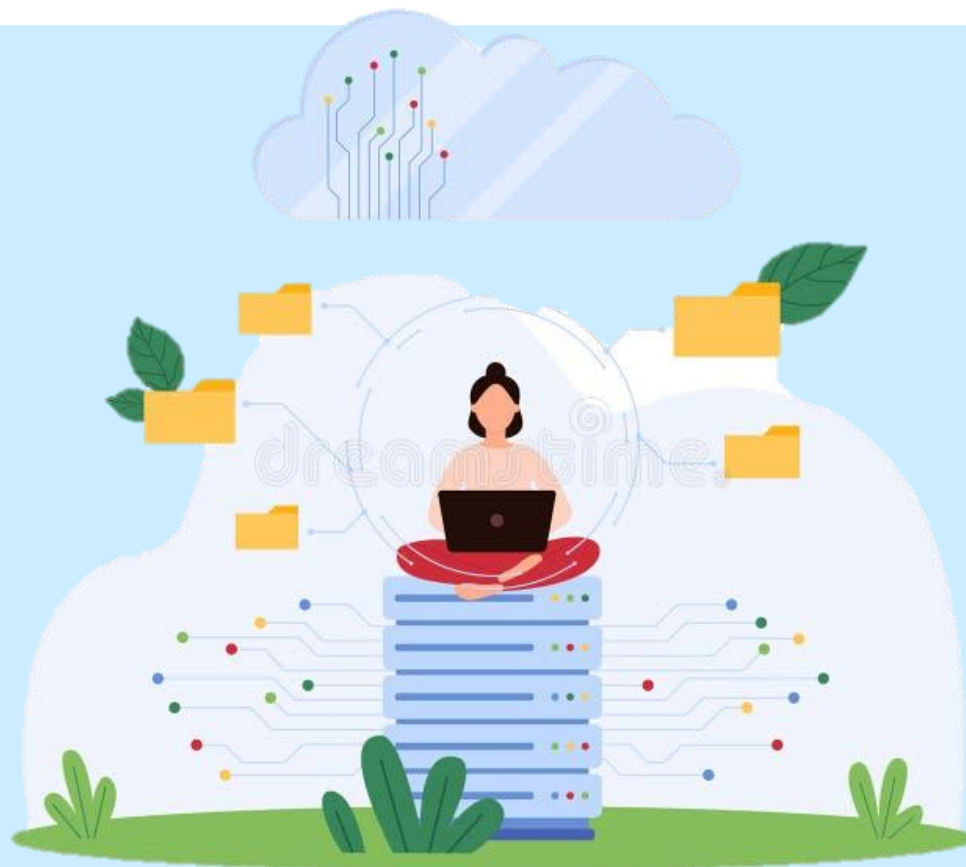
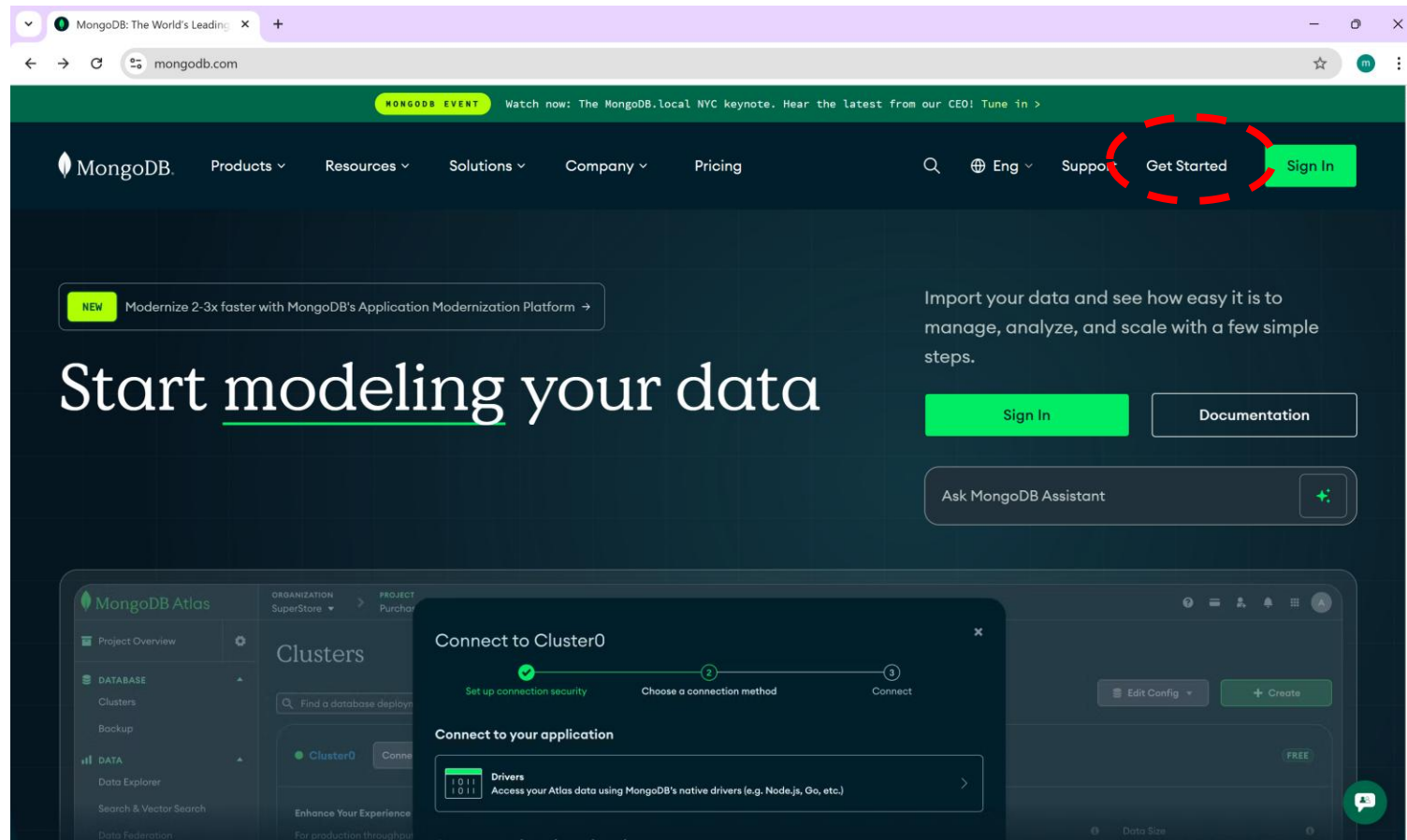


การสร้าง Node.js ติดต่อ MongoDB



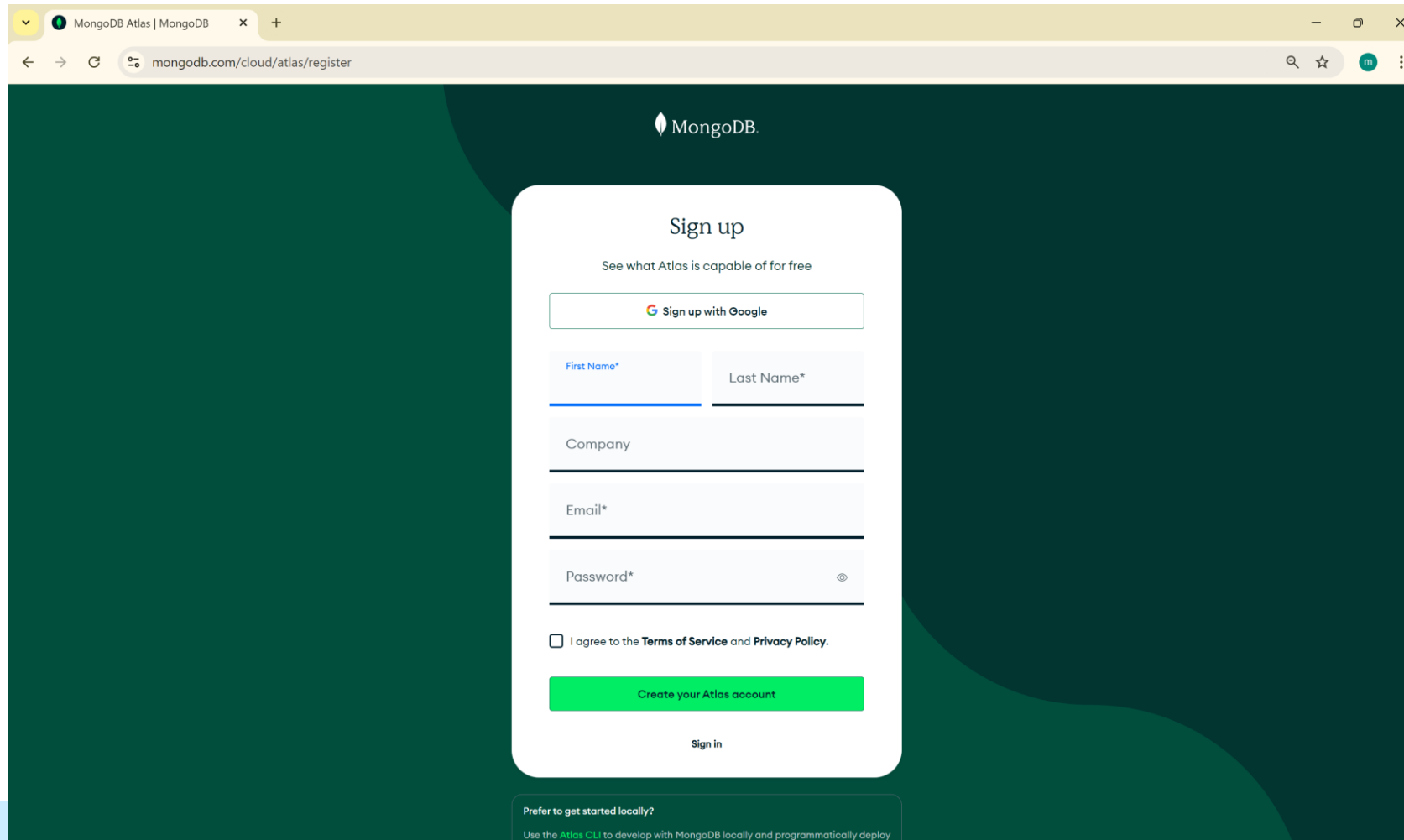
MongoDB Atlas.

- MongoDB Atlas คือ ระบบ MongoDB as a service ที่เป็นการรวมกันของ MongoDB เอง กับ Atlas ที่เป็นผู้ให้บริการ Cloud <https://www.mongodb.com/>



ลงทะเบียน MongoDB Atlas.

- สมัครสมาชิก <https://www.mongodb.com/cloud/atlas/register>



The screenshot shows the MongoDB Atlas registration page in a web browser. The browser's address bar displays the URL `mongodb.com/cloud/atlas/register`. The page features a dark green background with a white registration form in the center. The form is titled "Sign up" and includes the subtext "See what Atlas is capable of for free". It offers a "Sign up with Google" button, followed by input fields for "First Name*", "Last Name*", "Company", "Email*", and "Password*". Below these fields is a checkbox for "I agree to the Terms of Service and Privacy Policy." and a prominent green button labeled "Create your Atlas account". A "Sign in" link is positioned at the bottom of the form. At the very bottom of the page, there is a link for "Prefer to get started locally?" with a brief description of the Atlas CLI.

MongoDB

Sign up

See what Atlas is capable of for free

 Sign up with Google

First Name* Last Name*

Company

Email*

Password* 

☐ I agree to the [Terms of Service](#) and [Privacy Policy](#).

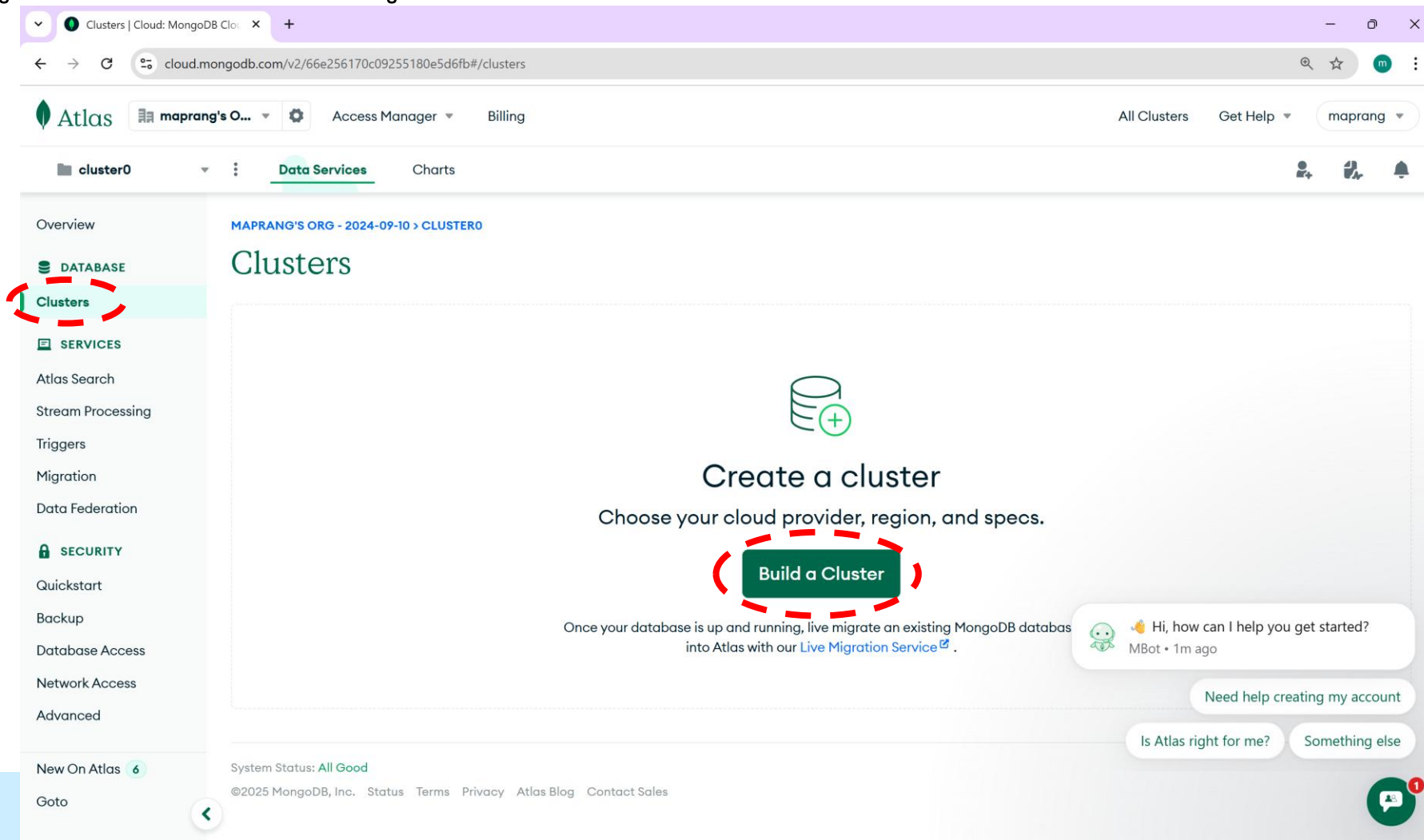
Create your Atlas account

[Sign in](#)

Prefer to get started locally?
Use the [Atlas CLI](#) to develop with MongoDB locally and programmatically deploy

สร้าง Cluster

- เมื่อเข้าสู่ระบบแล้วให้เลือกเมนู Cluster แล้วเลือก Build a Cluster



สร้าง Cluster

Deploy your cluster | Cloud: M

cloud.mongodb.com/v2/66e256170c09255180e5d6fb#/clusters/starterTemplates

Deploy your cluster

Use a template below or set up advanced configuration options. You can also edit these configuration options once the cluster is created.

☐ M10

\$0.09/hour

Dedicated cluster for development environments and low-traffic applications.

STORAGE	RAM	vCPU
10 GB	2 GB	2 vCPUs

☐ Flex

From \$0.011/hour
Up to \$30/month

For development and testing, with on-demand burst capacity for unpredictable traffic.

STORAGE	RAM	vCPU
5 GB	Shared	Shared

☒ Free

For learning and exploring MongoDB in a cloud environment.

STORAGE	RAM	vCPU
512 MB	Shared	Shared

✔ Free forever! Your free cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime.

Configurations

Name

You cannot change the name once the cluster is created.

Cluster0

Provider

aws

Google Cloud

Azure

Region

Singapore (ap-southeast-1) ★

★ Recommended ⓘ

Low carbon emissions ⓘ

Tag (optional)

Create your first tag to categorize and label your resources; more tags can be added later. [Learn more.](#)

Quick setup

☒ Automate security setup ⓘ

☒ Preload sample dataset ⓘ

I'll do this later

Go to Advanced Configuration

Create Deployment

Hi, how can I help you get started?
MBot • 4m ago

Need help creating my account

Is Atlas right for me?

Something else

1

ใช้ MongoDB Driver เพื่อเชื่อมต่อกับฐานข้อมูล MongoDB โดยตรง

Project Overview | Cloud: Mong x +

cloud.mongodb.com/v2/66e256170c09255180e5d6fb#/overview?automateSecurity=true&connectCluster=Cluster0

Connect to Cluster0

Set up connection security (1) Choose a connection method (2) Connect (3)

Connect to your application

Drivers
Access your Atlas data using MongoDB's native drivers (e.g. Node.js, Go, etc.)

Access your data through tools

- Compass**
Explore, modify, and visualize your data with MongoDB's GUI
- Shell**
Quickly add & update data using MongoDB's Javascript command-line interface
- MongoDB for VS Code**
Work with your data in MongoDB directly from your VS Code environment
- Atlas SQL**
Easily connect SQL tools to Atlas for data analysis and visualization

Go Back Close

Overview

We are deploying your changes (current action: config)

MAPRANG'S ORG - 2024-09-10 > CLUSTER0

Overview

Clusters

cluster0

Connect Edit configuration

Add data

Load sample data

+ Add Tag

Application Development

JavaScript / Node.js

Connect your application

Start with a Node.js Sample App

Loading sample dataset
We are loading the sample dataset to Cluster0.

Hi, how can I help you get started?
MBot • 6m ago

Need help creating my account

Is Atlas right for me? Something else

Support Plan: Basic Plan

ใช้ MongoDB Driver เพื่อเชื่อมต่อกับฐานข้อมูล MongoDB โดยตรง

Project Overview | Cloud: Mong x +

cloud.mongodb.com/v2/66e256170c09255180e5d6fb#/overview?automateSecurity=true&connectCluster=Cluster0

Connect to Cluster0

Set up connection security Choose a connection method **3 Connect**

Connecting with MongoDB Driver

1. Select your driver and version
We recommend installing and using the latest driver version.

Driver	Version
Node.js	6.7 or later

2. Install your driver
Run the following on the command line

```
npm install mongodb
```

[View MongoDB Node.js Driver installation instructions.](#)

3. Add your connection string into your application code
Use this connection string in your application

☐ View full code sample

```
mongodb+srv://mootigger:<db_password>@cluster0.d1ftea.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0
```

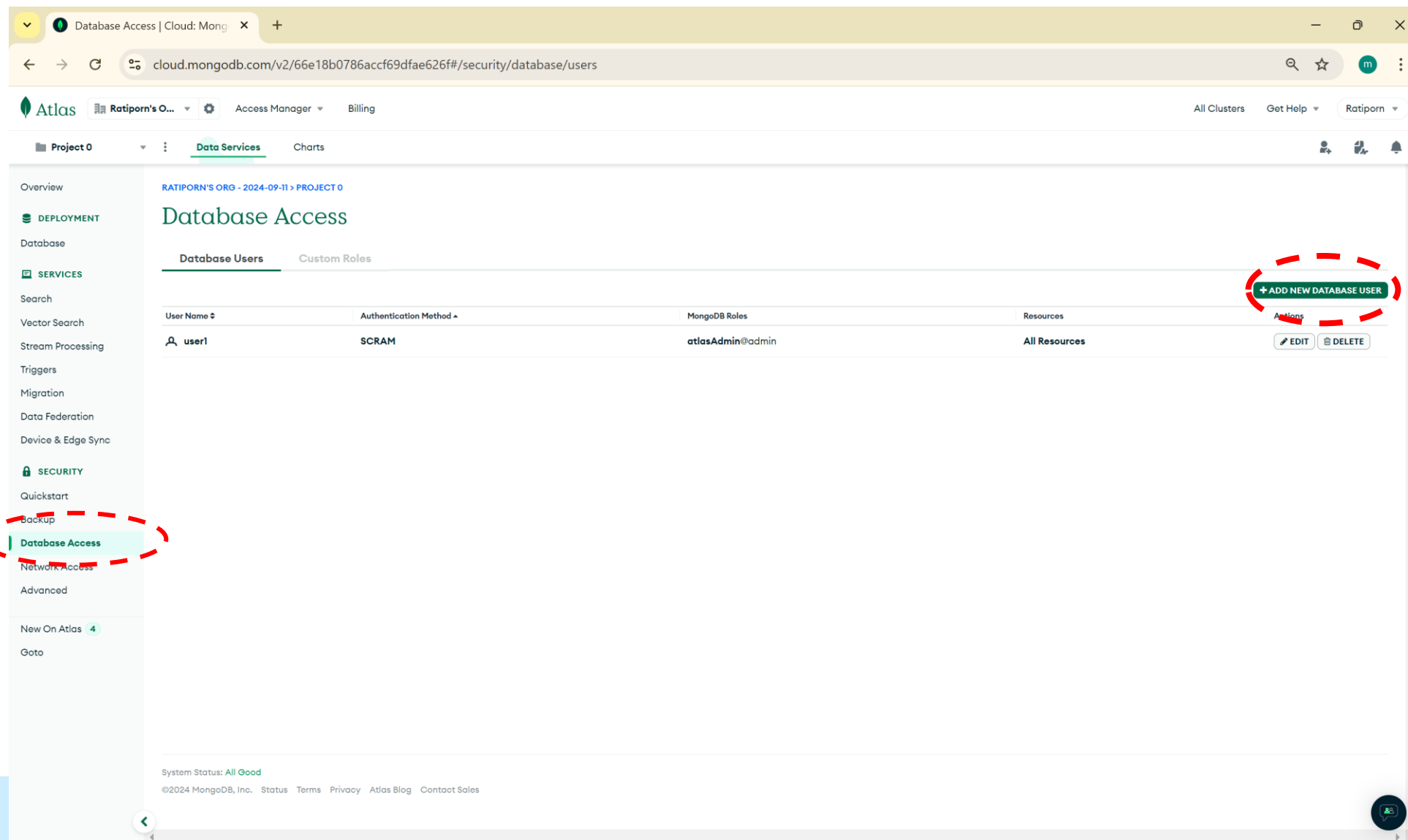
Replace **<db_password>** with the password for the **mootigger** database user. Ensure any option params are [URL encoded](#).

RESOURCES

- [Get started with the Node.js Driver](#)
- [Node.js Starter Sample App](#)
- [Access your Database Users](#)
- [Troubleshoot Connections](#)

[Go Back](#) [Done](#)

สามารถดู user ที่ใช้งานใน Database ได้



The screenshot displays the MongoDB Atlas interface for a project named "Ratiporn's Org". The "Database Access" section is highlighted in the left sidebar. The main content area shows the "Database Users" tab, which contains a table of users. A red dashed circle highlights the "+ ADD NEW DATABASE USER" button in the top right corner of the table. Another red dashed circle highlights the "Database Access" item in the left sidebar.

Database Access | Cloud: Mong

cloud.mongodb.com/v2/66e18b0786accf69dfe626f#/security/database/users

Atlas Ratiporn's Org Access Manager Billing

Project 0 Data Services Charts

Overview

DEPLOYMENT

Database

SERVICES

Search

Vector Search

Stream Processing

Triggers

Migration

Data Federation

Device & Edge Sync

SECURITY

Quickstart

Backup

Database Access

Network Access

Advanced

New On Atlas 4

Goto

RATIPORN'S ORG - 2024-09-11 > PROJECT 0

Database Access

Database Users Custom Roles

User Name	Authentication Method	MongoDB Roles	Resources	Actions
user1	SCRAM	atlasAdmin@admin	All Resources	EDIT DELETE

System Status: All Good

©2024 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

สร้าง username และ password สำหรับใช้ใน Database นี้

The screenshot shows the 'Add New Database User' page in the MongoDB Atlas interface. The page is titled 'Add New Database User' and provides instructions on creating a database user. The 'Authentication Method' section shows 'Password' selected. The 'Password Authentication' section shows the username 'user1' and the password 'AAAA'. The 'Built-in Role' dropdown is open, showing 'Atlas admin', 'Read and write to any database', and 'Only read any database'. The 'Database User Privileges' section is also visible.

Add New Database User

Create a database user to grant an application or user access to databases and collections in your clusters in this Atlas project. Granular access control can be configured with default privileges or custom roles. You can grant access to an Atlas project or organization using the corresponding [Access Manager](#).

Authentication Method

Password Certificate AWS IAM Federated Auth (MongoDB 7.0 and up)

MongoDB uses **SCRAM** as its default authentication method.

Password Authentication

user1

AAAA HIDE

Autogenerate Secure Password Copy

User Description

Add an optional description to your user.

Database User Privileges

Configure role based access control by assigning database user a mix of one built-in role, multiple custom roles, and multiple specific privileges. A user will gain access to all actions within the roles assigned to them, not just the actions those roles share in common. You must choose at least one role or privilege. [Learn more about roles.](#)

Built-in Role 0 SELECTED

Select one built-in role for this user.

Select Role

Atlas admin

Read and write to any database

Only read any database

Specific Privileges

Select multiple privileges and what database and collection they are associated with.

Database

The screenshot displays the MongoDB Atlas interface for a specific cluster. The left sidebar contains navigation options, with 'Database' highlighted. The main panel shows the 'Clusters' page for 'RATIPORN'S ORG - 2024-09-11 > PROJECT 0'. A green banner at the top indicates that a sample dataset was successfully loaded. Below this, there are buttons for 'Cluster0', 'Connect', 'View Monitoring', and 'Browse Collections'. A section titled 'Enhance Your Experience' provides a summary of cluster metrics: Read (R) at 0, Write (W) at 0, Connections at 0, Ingestion (In) at 0.0 B/s, and Output (Out) at 0.0 B/s. It also shows the Data Size as 82.0 MB / 512.0 MB (16%). A table at the bottom lists cluster details including Version (7.0.12), Region (AWS / Singapore), Cluster Tier (M0 Sandbox), Type (Replica Set - 3 nodes), Backups (Inactive), Linked App Services (None Linked), Atlas SQL (Connect), and Atlas Search (Create Index). The footer shows the system status as 'All Good' and copyright information for MongoDB, Inc.

Atlas Ratiporn's O... Access Manager Billing All Clusters Get Help Ratiporn

Project 0 Data Services Charts

Overview

RATIPORN'S ORG - 2024-09-11 > PROJECT 0

Clusters

Find a database deployment...

Edit Config + Create

Sample dataset successfully loaded. Access it in [Collections](#) or by connecting with the MongoDB Shell.

Cluster0 Connect View Monitoring Browse Collections ... FREE SHARED

Enhance Your Experience

For production throughput and richer metrics, upgrade to a dedicated cluster now!

Upgrade

R 0 W 0 Last 26 minutes 328.0/s

Connections 0 Last 26 minutes 1.0

In 0.0 B/s Out 0.0 B/s Last 26 minutes 599.0 KB/s

Data Size 82.0 MB / 512.0 MB (16%) Last 26 minutes 512.0 MB

VERSION	REGION	CLUSTER TIER	TYPE	BACKUPS	LINKED APP SERVICES	ATLAS SQL	ATLAS SEARCH
7.0.12	AWS / Singapore (ap-southeast-1)	M0 Sandbox (General)	Replica Set - 3 nodes	Inactive	None Linked	Connect	Create Index

+ Add Tag

System Status: All Good

©2024 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

วิธีการดูการเชื่อมต่อ

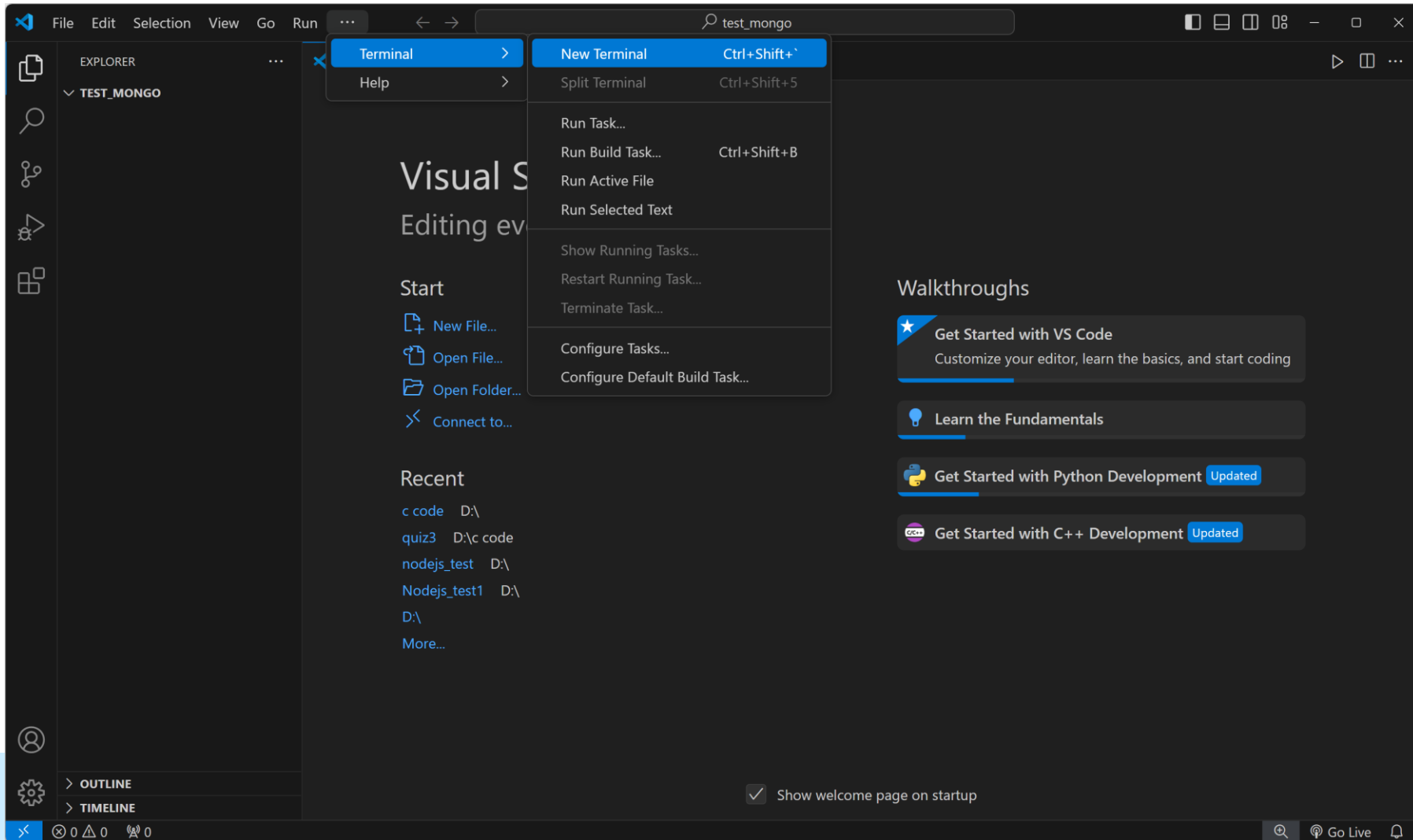
The screenshot shows the MongoDB Atlas interface. In the left sidebar, the 'Database' tab is highlighted with a red dashed circle. The main content area displays the 'Clusters' page for 'RATIPORN'S ORG - 2024-09-11 > PROJECT 0'. A green banner at the top indicates 'Sample dataset successfully loaded. Access it in [Collections](#) or by connecting with the MongoDB Shell.' Below this, the 'Cluster0' cluster is shown with buttons for 'Connect', 'View Monitoring', and 'Browse Collections'. The 'Cluster0' label is also circled with a red dashed circle. The cluster details section includes a table with the following data:

VERSION	REGION	CLUSTER TIER	TYPE	BACKUPS	LINKED APP SERVICES	ATLAS SQL	ATLAS SEARCH
7.0.12	AWS / Singapore (ap-southeast-1)	M0 Sandbox (General)	Replica Set - 3 nodes	Inactive	None Linked	Connect	Create Index

Below the table, there is a '+ Add Tag' button. The bottom of the page shows the system status as 'All Good' and the copyright notice '©2024 MongoDB, Inc.'.

เปิด Visual Studio Code

- สร้าง folder สำหรับเก็บงาน แล้วเลือก Terminal



การติดตั้งไลบรารี

- สร้างไฟล์ package.json ด้วยคำสั่ง `npm init -y`
- ติดตั้ง Express.js ได้ผ่านคำสั่ง `npm install express`
- ติดตั้ง Nodemon ช่วย Restart node server ให้โดยอัตโนมัติโดยใช้คำสั่ง `npm install --save-dev nodemon`
- ติดตั้ง CORS packages ด้วยคำสั่ง `npm install cors`
- ติดตั้ง Joi ผ่าน: `npm install joi` สำหรับการตรวจสอบและตรวจสอบความถูกต้องของข้อมูลใน Node.js ซึ่งมักใช้ในการตรวจสอบข้อมูลที่รับมาจากการร้องขอ (requests) หรือข้อมูลที่ใช้กรอกเข้ามา เพื่อให้แน่ใจว่าข้อมูลมีรูปแบบและค่าที่ถูกต้องตามที่ต้องการ
- ติดตั้ง createValidator ใช้สร้างตัวตรวจสอบ (validator) ที่สามารถใช้ในการตรวจสอบข้อมูลที่ได้รับจาก request ผ่าน: `npm install express joi express-joi-validation`
- ติดตั้ง mongoose ด้วยคำสั่ง `npm install mongoose`
- ติดตั้ง dotenv ด้วยคำสั่ง `npm install dotenv`
- สามารถใช้งาน Web Server ของเราได้ด้วยการใช้คำสั่ง `npx nodemon index.js` หลังจากที่เรารันโปรแกรมแล้ว Cursor บน Command line จะค้างอยู่แบบนั้นตลอดไป นั่นเป็นเพราะว่าเว็บเซิร์ฟเวอร์กำลังรอรับคำร้องขออยู่แบบ Asynchronous และมันจะทำงานต่อไปจนกว่าจะมีการกดปุ่ม CTRL+C เพื่อออกจากโปรแกรม
- ใส่ URL ใน Web Browser `http://localhost:3000/` (ใช้ตามที่กำหนด port)

- สร้างไฟล์ index.js ขึ้นมาตรง Path เดียวกันกับ package.json
- สร้างไฟล์ .env ใช้เก็บค่าต่างๆ เช่น API keys, รหัสผ่าน, พอร์ต, หรือการตั้งค่าอื่นๆ ที่ไม่ต้องการให้เผยแพร่ในโค้ดลงในไฟล์
- สร้างโฟลเดอร์ src ภายในประกอบด้วย

- index.js ไฟล์หลักที่เริ่มต้นการทำงานของเซิร์ฟเวอร์หรือแอปพลิเคชัน
- mongoose.db.js
- product.route.js
- product.schema.js ไฟล์ที่ใช้จัดการ schema ของ product
- product.service.js ไฟล์ที่ใช้จัดการการเชื่อมต่อฐานข้อมูล product สร้างฟังก์ชันเพิ่มข้อมูล
- product.controller.js ไฟล์ที่ใช้สร้างฟังก์ชันรองรับการทำงานของไฟล์ product.route.js
- create-product.dto.js
- request-info.middleware.js
- index.route.js
- index.middleware.js
- index.config.js

กำหนดโครงสร้าง การจัดเก็บไฟล์

ไฟล์ index.js

- สร้างไฟล์ index.js เป็นไฟล์จุดเริ่มต้นของแอปพลิเคชัน ซึ่งเป็นไฟล์หลักที่เริ่มต้นการทำงานของเซิร์ฟเวอร์หรือแอปพลิเคชัน
- สร้างไฟล์ package.json ด้วยคำสั่ง `npm init -y`
- ใช้ `import express from 'express'` และอย่าลืมเพิ่ม `"type": "module"` ถัดจาก `"main": "index.js"` ใน package.json (เป็นการเขียนแบบ ESM (ECMAScript Modules))
- ติดตั้ง Express.js ได้ผ่านคำสั่ง `npm install express`
- ติดตั้ง Nodemon ช่วย Restart node server ให้โดยอัตโนมัติโดยใช้คำสั่ง `npm install nodemon`
- สามารถใช้งาน Web Server ของได้ด้วยการใช้คำสั่ง `npx nodemon index.js` หลังจากที่เรารันโปรแกรมแล้ว Cursor บน Command line จะค้างอยู่แบบนั้นตลอดไป นั่นเป็นเพราะว่าเว็บเซิร์ฟเวอร์กำลังรอรับคำร้องขออยู่แบบ Asynchronous และมันจะทำงานตลอดไปจนกว่าจะมีการกดปุ่ม CTRL+C เพื่อออกจากโปรแกรม
- ใส่ URL ใน Web Browser `http://localhost:3000/` (ใช้ตามที่กำหนด port)

ไฟล์ index.js

```
import express from 'express';
const app = express();
app.get('/', (req, res) => {
  res.status(200).json({
    success: true,
    date: {
      timestamp: new Date()
    }
  })
})
const PORT = 3000
app.listen(PORT, () => {
  console.log(`http://localhost:${PORT}`);
});
```

ไฟล์ index.middleware.js

- ติดตั้ง Express.js ได้ผ่านคำสั่ง `npm install express`
- ติดตั้ง CORS packages ด้วยคำสั่ง `npm install cors`
- สร้างไฟล์ `index.middleware.js` เป็นไฟล์ที่เก็บรวบรวม middleware ที่ใช้งานทั้งหมด
- ในโค้ด มี เรียกใช้งาน CORS เพื่อให้โปรเจกอนุญาตให้เข้าถึงแหล่งข้อมูลต่างโดเมนได้
- แล้ว import ไปใช้ใน `index.js` เพื่อให้ระบบนำไปใช้งาน

ไฟล์ index.middleware.js

```
import express from 'express'
import cors from 'cors'
const IndexMiddleware = express()
IndexMiddleware.use(cors())
IndexMiddleware.use(express.urlencoded({extended:true}))
IndexMiddleware.use(express.json())
export default IndexMiddleware
```



ไฟล์ index.js

```
import express from 'express';
import IndexMiddleware from './index.middleware.js';
const app = express();
app.use(IndexMiddleware)
app.get('/',(req,res)=>{
  res.status(200).json({
    success: true,
    date: {
      timestamp: new Date()
    }
  })
})
const PORT = 3000
app.listen(PORT, () => {
  console.log(`http://localhost:${PORT}`);
});
```

สร้างไฟล์ request-info.middleware.js

- สร้างไฟล์ request-info.middleware.js เพื่อสร้าง Middleware สำหรับ log ข้อมูลทุกครั้งที่ Request มาจะแสดง url และ method ออกมาแสดง
- แล้ว import ไปใช้ใน index.js เพื่อให้ระบบนำไปใช้งาน

ไฟล์ request-info.middleware.js

```
const RequestMiddleware = function(req, res, next) {  
  console.info(`[${new Date()}]: { URL: ${req.originalUrl}, Method: ${req.method} }`)  
  next()  
}  
  
export default RequestMiddleware
```



ไฟล์ index.middleware.js

```
import express from 'express'  
import cors from 'cors'  
import RequestMiddleware from './request-info.middleware.js'  
const IndexMiddleware = express()  
IndexMiddleware.use(cors())  
IndexMiddleware.use(RequestMiddleware)  
IndexMiddleware.use(express.urlencoded({extended:true}))  
IndexMiddleware.use(express.json())  
  
export default IndexMiddleware
```

ไฟล์ .env

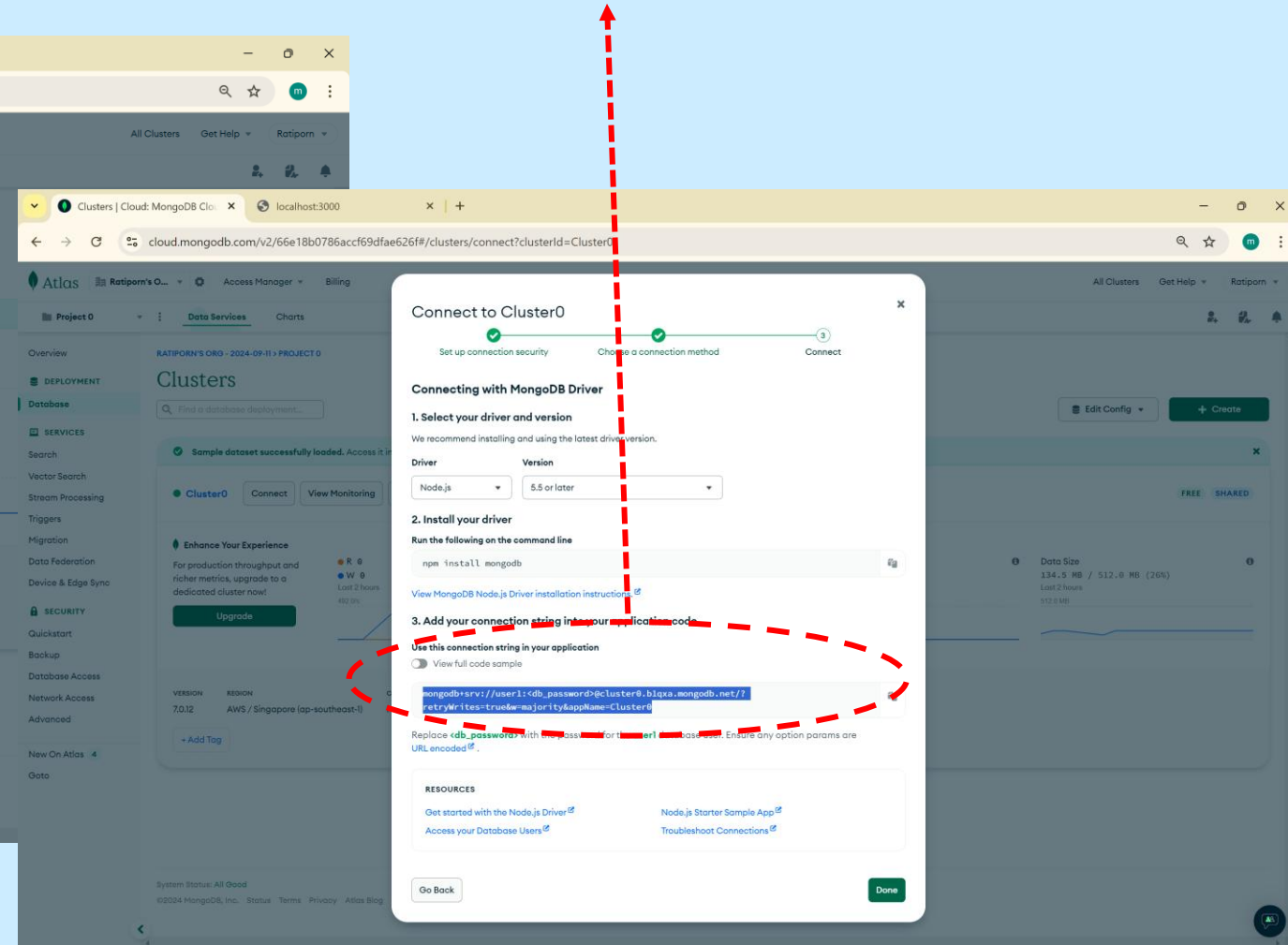
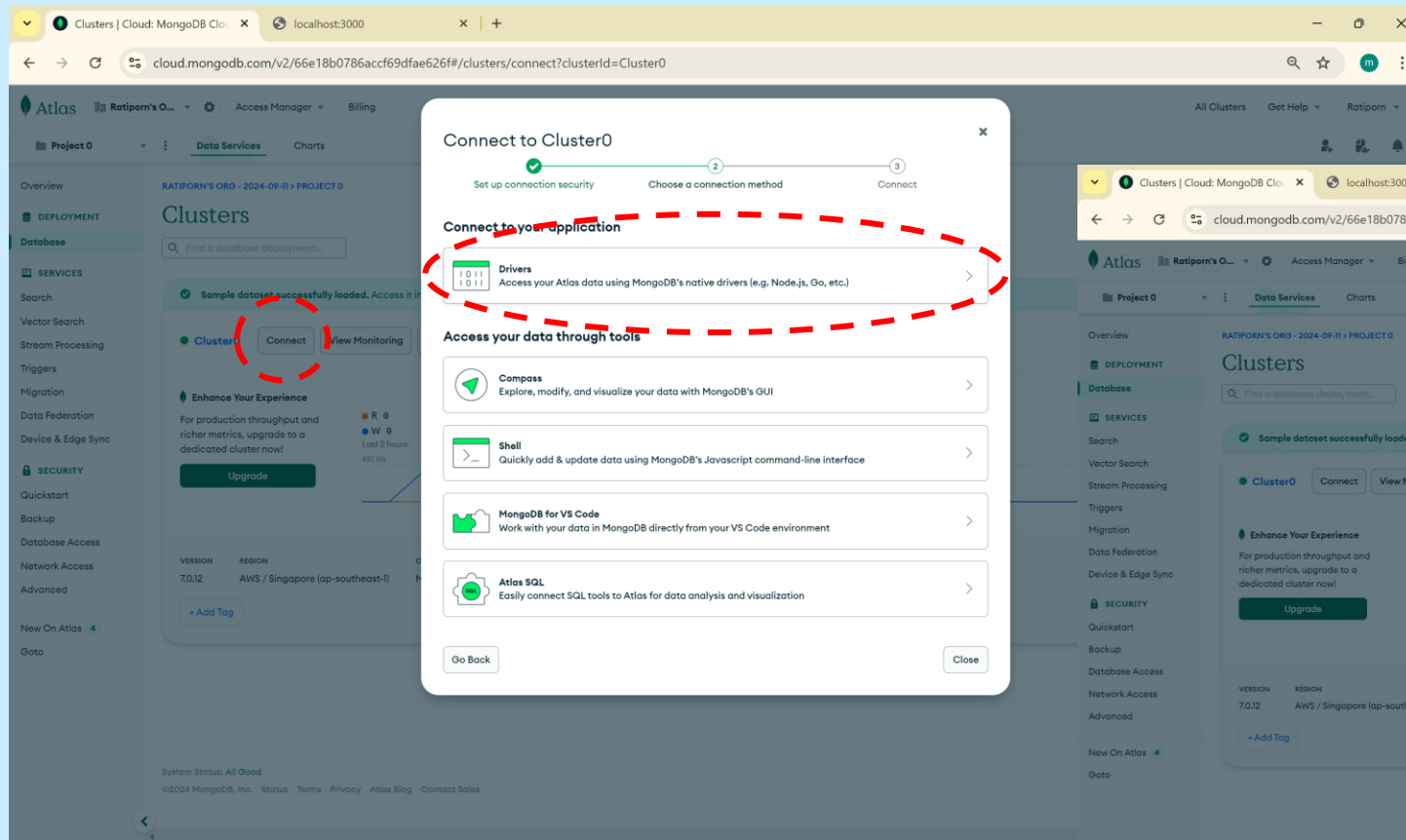
- ไฟล์ .env (environment file) คือไฟล์ที่ใช้เก็บ ค่าตัวแปรสภาพแวดล้อม (environment variables) ของโปรเจกต์ เช่น รหัสผ่าน, คีย์ลับ, URL ของฐานข้อมูล, พอร์ตเซิร์ฟเวอร์ ฯลฯ
- จุดประสงค์หลัก
 - แยกข้อมูลที่ เปลี่ยนบ่อย/สำคัญ ออกจากโค้ด (เช่น connection string ของ MongoDB, API key)
 - ป้องกันไม่ให้ข้อมูลลับ (เช่น password) ไปอยู่ในโค้ดโดยตรง
 - ทำให้การ deploy ไปหลาย environment (development, test, production)ง่ายขึ้น เพราะเปลี่ยนค่าแค่ใน .env ไม่ต้องแก้โค้ด
- ความปลอดภัย
 - ห้าม commit ไฟล์ .env ขึ้น GitHub → ควรใส่ชื่อ .env ในไฟล์ .gitignore
 - ถ้าเป็น production ควรกำหนด environment variables ผ่าน server หรือ service เช่น Docker, Kubernetes, Vercel, Heroku แทนการใช้ไฟล์ .env
- โครงสร้างไฟล์ .env
 - เป็นไฟล์ข้อความธรรมดา (ไม่ต้องใส่ JSON หรือ YAML) ใช้รูปแบบ KEY=VALUE

ไฟล์ .env ในกรณีใช้ MongoDB Atlas.

username และ password

เพิ่มชื่อ database

```
MONGODB_CONNECTION=mongodb+srv://user1:AAAAA@cluster0.d1fteea.mongodb.net/myDB?retryWrites=true&w=majority&appName=Cluster0
```

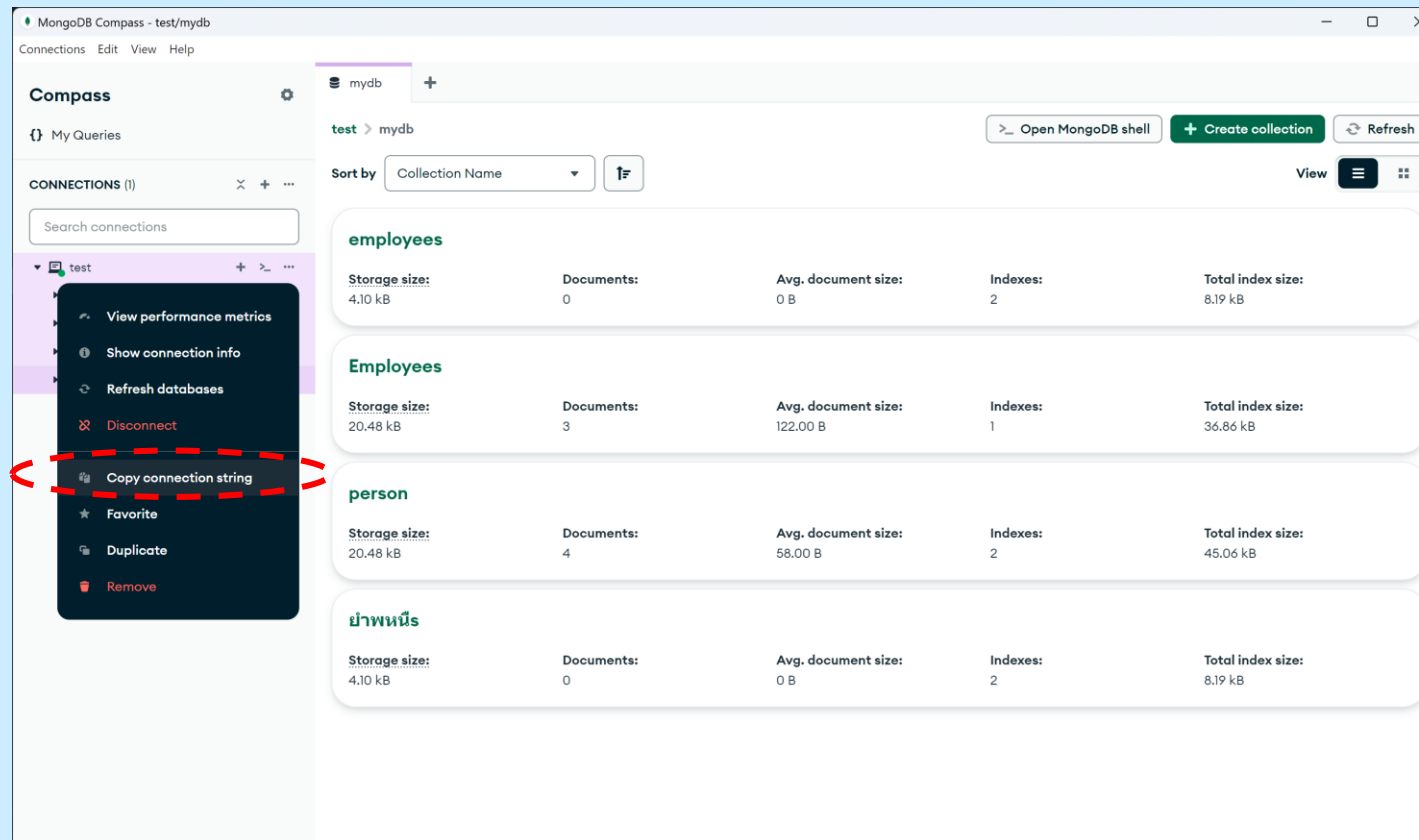


ไฟล์ .env ในกรณีใช้ MongoDB Compass

เพิ่มชื่อ database

- กดเมนู Copy connection string จะได้ mongodb://localhost:27017/mydb คือ Connection String ที่เชื่อมต่อกับ MongoDB ที่รันอยู่ในเครื่อง local และฐานข้อมูลชื่อว่า mydb

MONGODB_CONNECTION=mongodb://localhost:27017/mydb



ไฟล์ mongoose.db.js

- สร้างไฟล์ mongoose.db.js เพื่อใช้ Mongoose เชื่อมต่อกับ MongoDB โดยดึงค่า connection string มาจาก .env
- ติดตั้ง Mongoose ผ่าน: `npm install mongoose`
 - ใช้เพื่อเชื่อมต่อกับ MongoDB
- ติดตั้ง dotenv ผ่าน: `npm install dotenv`
 - เพื่อใช้ในการโหลดไฟล์ .env (ที่เก็บตัวแปร environment เช่น password, API key, connection string) แล้วเอาค่าพวกนั้นมาใส่ใน process.env ของ Node.js ให้ใช้งานได้
- dotenv.config() เป็นคำสั่งที่ใช้ใน Node.js เพื่อให้แพ็คเกจ dotenv โหลดข้อมูลจากไฟล์ .env และตั้งค่า environment variables ในระบบตามที่กำหนดไว้ในไฟล์นั้น
- mongoose.connect(process.env.MONGODB_CONNECTION)
 - ใช้เพื่อเชื่อมต่อกับ MongoDB ต้องใส่ Connection String (เช่น "mongodb://localhost:27017/mydb" หรือ URL จาก MongoDB Atlas)
 - หมายถึงตัวแปร environment ชื่อ MONGODB_CONNECTION ที่เก็บไว้ในไฟล์ .env

ไฟล์ mongoose.db.js

```
import mongoose from "mongoose";
import dotenv from "dotenv";

dotenv.config(); // โหลดค่าจาก .env

mongoose.connect(process.env.MONGODB_CONNECTION)
  .then(() => console.log("Connected to MongoDB"))
  .catch(err => console.error("Failed to connect to MongoDB", err));

export default mongoose
```

ไฟล์ index.config.js

- เป็นไฟล์จัดการการตั้งค่าการเชื่อมต่อฐานข้อมูลและการตั้งค่าอื่น ๆ สำหรับแอปพลิเคชัน
- เรียกใช้การเชื่อมต่อฐานข้อมูล MongoDB จาก mongoose.db.js

```
import express from 'express'  
import './mongoose.db.js'  
  
const IndexConfig = express()  
export default IndexConfig
```

ไฟล์ index.config.js

ไฟล์ index.js

ไฟล์ index.js

- เพิ่มการเรียกใช้ไฟล์ index.config.js ลงใน ไฟล์ index.js

```
import express from 'express';
import IndexMiddleware from './index.middle.js';
import IndexConfig from './index.config.js';
const app = express();
app.use(IndexConfig);
app.use(IndexMiddleware);
app.get('/', (req, res) => {
  res.status(200).json({
    success: true,
    date: {
      timestamp: new Date()
    }
  })
});
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`http://localhost:${PORT}`);
});
```

```
PS C:\Users\Lenovo\Desktop\test2> npx nodemon index.js
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
[dotenv@17.2.2] injecting env (1) from .env -- tip: ⚙️
http://localhost:3000
Connected to MongoDB
```

Schema

- Schema = แบบแผน (template) ของข้อมูล
- ใช้กำหนดว่า Collection (ตารางใน MongoDB) จะมี field อะไรบ้าง, ชนิดข้อมูล (String, Number, Date ฯลฯ), ค่าบังคับ, ค่า default ฯลฯ
- ใน Mongoose กำหนด Schema ผ่าน `new mongoose.Schema({...})`
- เช่น

```
const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true, trim: true },
  email: { type: String, required: true, lowercase: true, match: /.+\@.+\..+/, },
  password: { type: String, required: true, minlength: 6, select: false },
  role: { type: String, enum: ["user", "admin"], default: "user" },
  age: { type: Number, min: 0, max: 120 },
  createdAt: { type: Date, default: Date.now, immutable: true }
});
```

Schema Options (ที่ใช้ง่าย)

Option	ใช้กับ	ความหมาย / การทำงาน	ตัวอย่าง
type	ทุก field	ชนิดข้อมูล (String, Number, Date, Boolean, Array, ObjectId)	{ type: String }
required	ทุก field	บังคับว่าต้องมีค่า	{ type: String, required: true }
unique	String, Number	ห้ามค่าซ้ำ (สร้าง unique index)	{ type: String, unique: true }
default	ทุก field	ค่าเริ่มต้นถ้าไม่ได้กรอก	{ type: Date, default: Date.now }
enum	String	จำกัดค่าให้เลือกเฉพาะเซตที่กำหนด	{ type: String, enum: ["user","admin"] }
min / max	Number, Date	ค่าต่ำสุด-สูงสุด	{ type: Number, min: 0, max: 100 }
match	String	ตรวจสอบด้วย regex	{ type: String, match: /.+\@.+\.+\/ }
trim	String	ตัดช่องว่างหัว-ท้าย	{ type: String, trim: true }
lowercase	String	แปลงค่าเป็นตัวเล็กเสมอ	{ type: String, lowercase: true }
uppercase	String	แปลงค่าเป็นตัวใหญ่เสมอ	{ type: String, uppercase: true }
validate	ทุก field	กำหนดเงื่อนไขตรวจสอบเอง	{ type: String, validate: v => v.length > 6 }
immutable	ทุก field	กำหนดว่าแก้ไขไม่ได้หลังสร้าง	{ type: Date, default: Date.now, immutable: true }
index	ทุก field	สร้าง index (ค้นหาเร็วขึ้น)	{ type: String, index: true }
select	ทุก field	เลือกที่จะ query field นี้มาด้วยไหม (default: true)	{ type: String, select: false }

- ใช้สร้าง Schema (หรือ "โครงสร้างข้อมูล") คือแนวทางหรือรูปแบบที่กำหนดโครงสร้างของข้อมูลซึ่งบ่งบอกว่าข้อมูลนั้นๆ จะถูกจัดเก็บและจัดระเบียบอย่างไร
- การสร้างและจัดการโมเดล (model) ของ Mongoose สำหรับการทำงานกับ MongoDB ใน Node.js โมเดลนี้ช่วยให้คุณสามารถทำการดำเนินการ CRUD (Create, Read, Update, Delete) กับคอลเล็กชัน products ได้
- ProductSchema: กำหนดโครงสร้างของเอกสาร (document) ที่จะจัดเก็บในคอลเล็กชัน products ในฐานข้อมูล MongoDB
 - name: เป็นฟิลด์ที่ต้องมีชนิดข้อมูลเป็นสตริง (String) และเป็นฟิลด์ที่จำเป็นต้องมี (required: true)
 - price: เป็นฟิลด์ที่มีชนิดข้อมูลเป็นตัวเลข (Number) และมีค่าเริ่มต้นเป็น 0 (default: 0)
 - timestamps: true: Mongoose จะสร้าง field พิเศษให้อัตโนมัติ 2 ตัว
 - createdAt → วันที่ document ถูกสร้าง
 - updatedAt → วันที่ document ถูกแก้ไขล่าสุด

ไฟล์ product.schema.js

```
import mongoose from "mongoose";
// 1. สร้าง Schema
const ProductSchema = new mongoose.Schema({
  name:{
    type: String,
    required: true
  },
  color:{
    type: String,
    default: 'NaN'
  },
  price:{
    type: Number,
    default: 0
  }
},{timestamps: true})

// 2. แปลง Schema เป็น Model
const ProductModel = mongoose.model("products", ProductSchema);
//products คือชื่อ collection

export default ProductModel;
```

ไฟล์ product.schema.js

เมธอดของ Mongoose

คำสั่ง	ทำอะไร
.find()	คืน array ของ document (แม้เจอแค่ 1 ตัวก็จะอยู่ใน array)
.findOne()	คืน document เดียว หรือ null ถ้าไม่เจอ
.findById(id)	ค้นหา document ตาม _id โดยตรง (สะดวกกว่าการเขียน { _id: ... })
.updateOne()	อัปเดต document แรก ที่ตรงเงื่อนไข
.updateMany()	อัปเดต ทุก document ที่ตรงเงื่อนไข
.findByIdAndUpdate()	หา document ด้วย _id แล้วอัปเดต (คืนค่า document)
.save()	ใช้กับ instance ของ document (insert หรือ update ได้)
.deleteOne()	ลบ document แรก ที่ตรงกับ filter
.deleteMany()	ลบ ทุก document ที่ตรงกับ filter
.findByIdAndDelete(id)	หา document ตาม _id แล้วลบ (คืนค่า document ที่ถูกลบ)

ไฟล์ product.service.js

- สร้างฟังก์ชันที่ใช้กำหนดการกระทำกับฐานข้อมูล

```
import ProductModel from "../product.schema.js";

// สร้าง service ชื่อ ProductService (เก็บฟังก์ชันที่ใช้จัดการข้อมูล product)
const ProductService = {
  // ฟังก์ชัน create สำหรับสร้างสินค้าใหม่
  create: (payload) => {
    // new ProductModel(payload) -> สร้าง object ใหม่จากข้อมูล payload
    // .save() -> บันทึก object นี้ลงใน MongoDB
    return new ProductModel(payload).save();
  }
}

export default ProductService;
```

ไฟล์ product.service.js

.save() (ของ Mongoose Document)

- .save() ใช้กับ instance ของ Model ที่สร้างจาก new Model()
- จะทำงานผ่าน Mongoose → มีการ validate ข้อมูลตาม Schema ก่อนเสมอ
- ใช้ได้ทั้ง insert (ถ้าเป็น document ใหม่) และ update (ถ้าเป็น document ที่โหลดมาจาก DB แล้วแก้ไข)

ไฟล์ product.controller.js

- เป็นไฟล์ที่ใช้กำหนดฟังก์ชันที่ใช้รองรับการทำงานของ route ในไฟล์ product.route.js

```
import ProductService from "../product.service.js"
const ProductController = {
  createProduct: async (req,res)=>{
    const{name,color,price} = req.body
    const create = await ProductService.create({name,color,price})

    res.status(201).json({
      success:true,
      data: create
    })
  }
}
export default ProductController
```

ไฟล์ product.controller.js

DTO

- DTO = Data Transfer Object = คลาส/อ็อบเจกต์ที่กำหนดโครงสร้างของข้อมูล ที่จะ รับ-ส่ง ระหว่างชั้นต่าง ๆ ของระบบ (Controller → Service → Database)
- ใช้เพื่อ
 - กำหนด รูปแบบข้อมูลชัดเจน (field อะไรบ้าง, ชนิดข้อมูล)
 - ช่วย validate ข้อมูลก่อนบันทึกลง DB
 - ลดความเสี่ยงที่ client จะส่ง field แปลก ๆ มาใส่ใน DB
- ไม่จำเป็นต้องมี DTO เสมอไป เพราะ Mongoose Schema ก็ทำหน้าที่ validate ได้ระดับหนึ่งแล้ว (เช่น required, min, max, match)
- ไม่จำเป็นต้องมี DTO ถ้าเป็นโปรเจกต์เล็ก หรือใช้ Mongoose Schema ตรวจสอบข้อมูลพอแล้ว
- ควรมี DTO ถ้าโปรเจกต์ใหญ่ ต้องการ validation ที่ชัดเจน, แยกชั้น Controller–Service–Model ออกชัด

ไฟล์ create-product.dto.js

- สร้าง DTO (ด้วย Joi schema)
- Joi = library สำหรับ validation ข้อมูล ใช้ Joi เพื่อสร้าง Schema → ทำหน้าที่ตรวจสอบว่า field ตรงตามที่ DTO ต้องการไหม
- สร้าง create-product.dto.js เพื่อใช้ในการ ตรวจสอบข้อมูล ที่ส่งเข้ามาใน request body เพื่อให้แน่ใจว่าข้อมูลเป็นไปตามรูปแบบที่กำหนดใน schema ของ Joi
- ติดตั้ง createValidator ใช้สร้างตัวตรวจสอบ (validator) ที่สามารถใช้ในการตรวจสอบข้อมูลที่ได้รับจาก request ผ่าน: `npm install joi express-joi-validation`

```
import Joi from 'joi'
// สร้าง Joi schema
export const CreateProductDto = Joi.object({
  name: Joi.string().required(), // 필드 name เป็น string และต้องส่งมา
  color: Joi.string().optional().default('NaN'), // 필드 color เป็น string และไม่บังคับส่งมา, มีค่าเริ่มต้นเป็น 'NaN' ถ้าไม่ระบุ
  price: Joi.number().optional().default(0) // 필드 price เป็น number, ไม่บังคับส่งมา และมีค่าเริ่มต้นเป็น 0 ถ้าไม่ระบุ
})
```

ไฟล์ create-product.dto.js

ไฟล์ product.route.js และ ไฟล์ index.route.js

- เป็นไฟล์ที่ใช้กำหนด route ในการเข้าถึง product
- ใช้ validator เป็น middleware ใน route โดยใช้ไฟล์ในโฟลเดอร์ dto ซึ่งตอนนี้คือ create-product.dto.js
- express-joi-validation = middleware สำหรับ Express.js ใช้ร่วมกับ Joi เพื่อ ตรวจสอบความถูกต้องของข้อมูล (validation) ที่มาจาก
 - req.body (ข้อมูลที่ client ส่งมาใน body)
 - req.query (ข้อมูล query string ?page=1)
 - req.params (ข้อมูล path parameter เช่น /users/:id)
 - req.headers (header ของ request)

ไฟล์ product.route.js

```
import express from 'express'
const ProductRouter = express.Router()
import ProductController from './product.controller.js'
import {createValidator} from 'express-joi-validation'
import { CreateProductDto } from './create-product.dto.js'
const validator = createValidator(); // สร้าง validator

ProductRouter.post('/', validator.body(CreateProductDto), ProductController.createProduct)

export default ProductRouter
```

ไฟล์ index.route.js

- สร้างไฟล์ index.route.js เพื่อรวม route ของ module ต่าง ๆ
- เพิ่มการเรียกใช้ไฟล์ product.route.js ลงใน ไฟล์ index.route.js

ไฟล์ index.route.js

```
import express from 'express'
import ProductRouter from './product.route.js';
const IndexRouter = express();

IndexRouter.use('/product',ProductRouter)

export default IndexRouter
```

ไฟล์ index.js

ไฟล์ index.js

- เพิ่มการเรียกใช้ไฟล์ index.route.js ลงใน ไฟล์ index.js

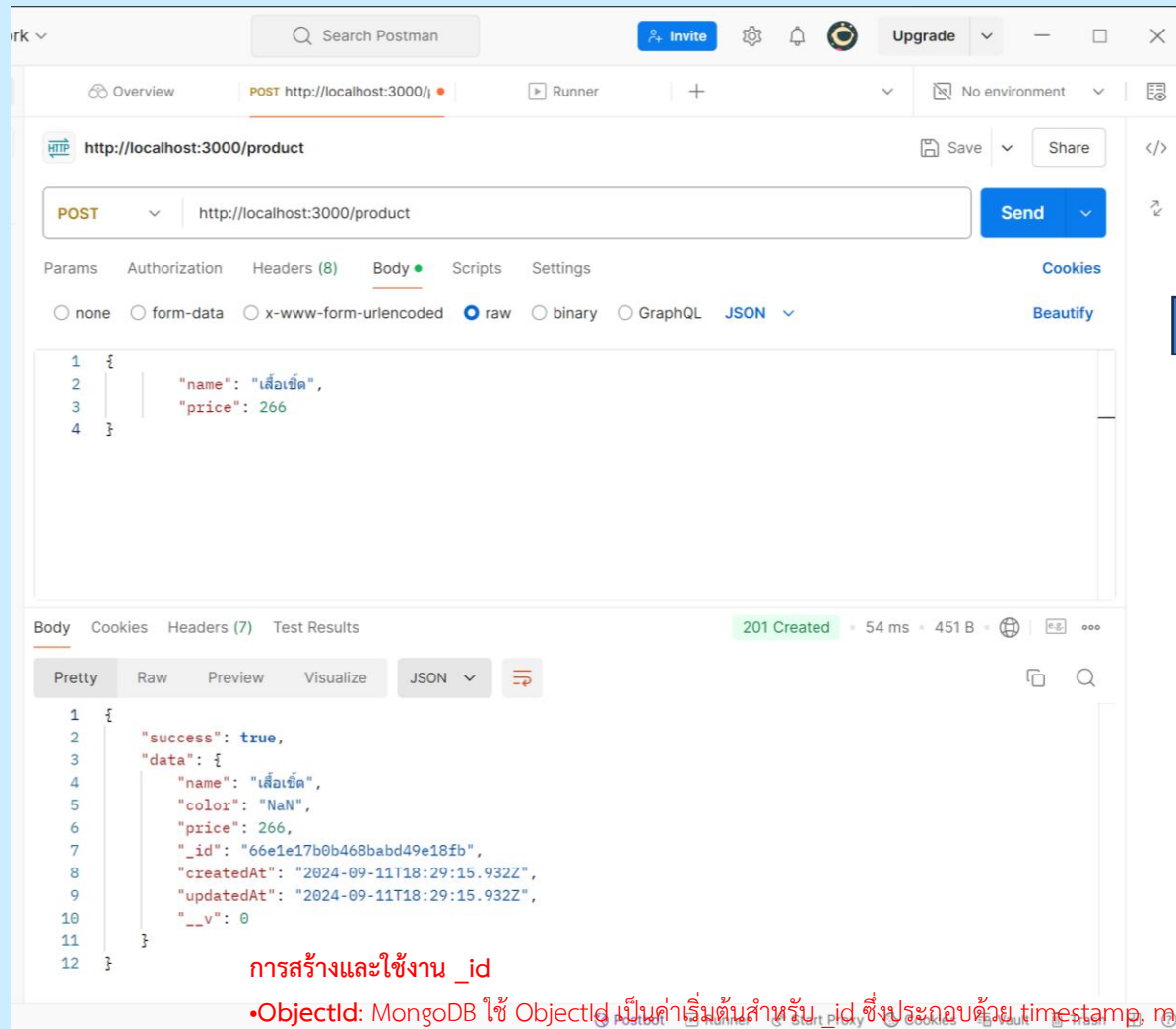
```
import express from 'express';
import cors from 'cors';
import IndexConfig from './src/index.config.js'
import IndexRouter from './src/index.route.js'
import IndexMiddleware from './src/index.middleware.js';
const app = express();
app.use(IndexConfig)
app.use(IndexMiddleware)

app.use(IndexRouter)

app.get('/', (req, res) => {
  res.status(200).json({
    success: true,
    date: {
      timestamp: new Date()
    }
  })
})

const PORT = 3000
app.listen(PORT, () => {
  console.log(`Server is running on port [${PORT}]`);
});
```

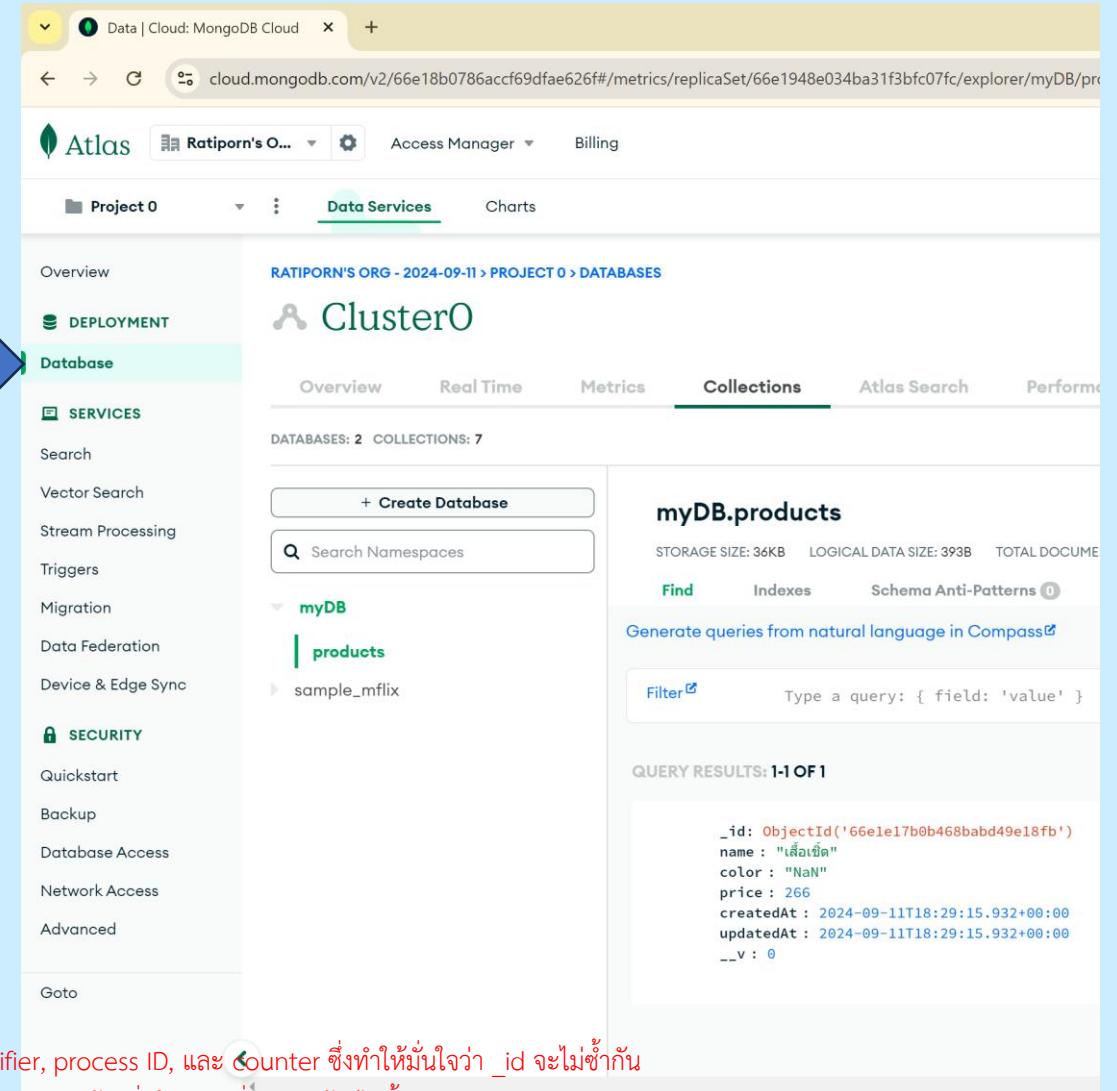
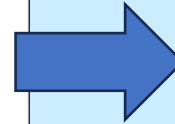
ทดลองใช้ postman ส่ง request โดย POST จะต้องมียข้อมูลเพิ่มใน MongoDB



การสร้างและใช้งาน `_id`

• **ObjectId**: MongoDB ใช้ ObjectId เป็นค่าเริ่มต้นสำหรับ `_id` ซึ่งประกอบด้วย timestamp, machine identifier, process ID, และ counter ซึ่งทำให้มั่นใจว่า `_id` จะไม่ซ้ำกัน

• **การกำหนด `_id` เอง**: แม้ว่า MongoDB จะสร้าง `_id` ให้เองโดยอัตโนมัติ, คุณสามารถกำหนด `_id` เองได้หากต้องการ แต่ต้องมั่นใจว่าค่าที่กำหนดต้องไม่ซ้ำกัน



ทำส่วนดึงข้อมูล product เพิ่ม

ไฟล์ product.service.js

```
import ProductModel from
"../models/product.schema.js"

const ProductService = {
  create: (payload) =>{
    return new ProductModel(payload).save()
  },
  getAll: (query = {})=>{
    return ProductModel.find(query)
  }
}
export default ProductService
```

ไฟล์ product.controller.js

```
import ProductService from "../services/product.service.js"
const ProductController = {
  createProduct: async (req,res)=>{
    const{name,color,price} = req.body
    const create = await ProductService.create({name,color,price})

    res.status(201).json({
      success:true,
      data: create
    })
  },
  getProduct : async (req,res)=>{
    const products = await ProductService.getAll()
    res.status(200).json({
      success:true,
      data: products
    })
  }
}
```

ไฟล์ product.route.js

```
import express from 'express'
const ProductRouter = express.Router()
import ProductController from '../product.controller.js'
import {createValidator} from 'express-joi-validation'
import { CreateProductDto } from '../dto/create-product.dto.js'

const validator = createValidator(); // สร้าง validator

ProductRouter .post('/',
validator.body(CreateProductDto),ProductController.createProduct)
ProductRouter .get('/', ProductController.getProduct)
export default ProductRouter
```

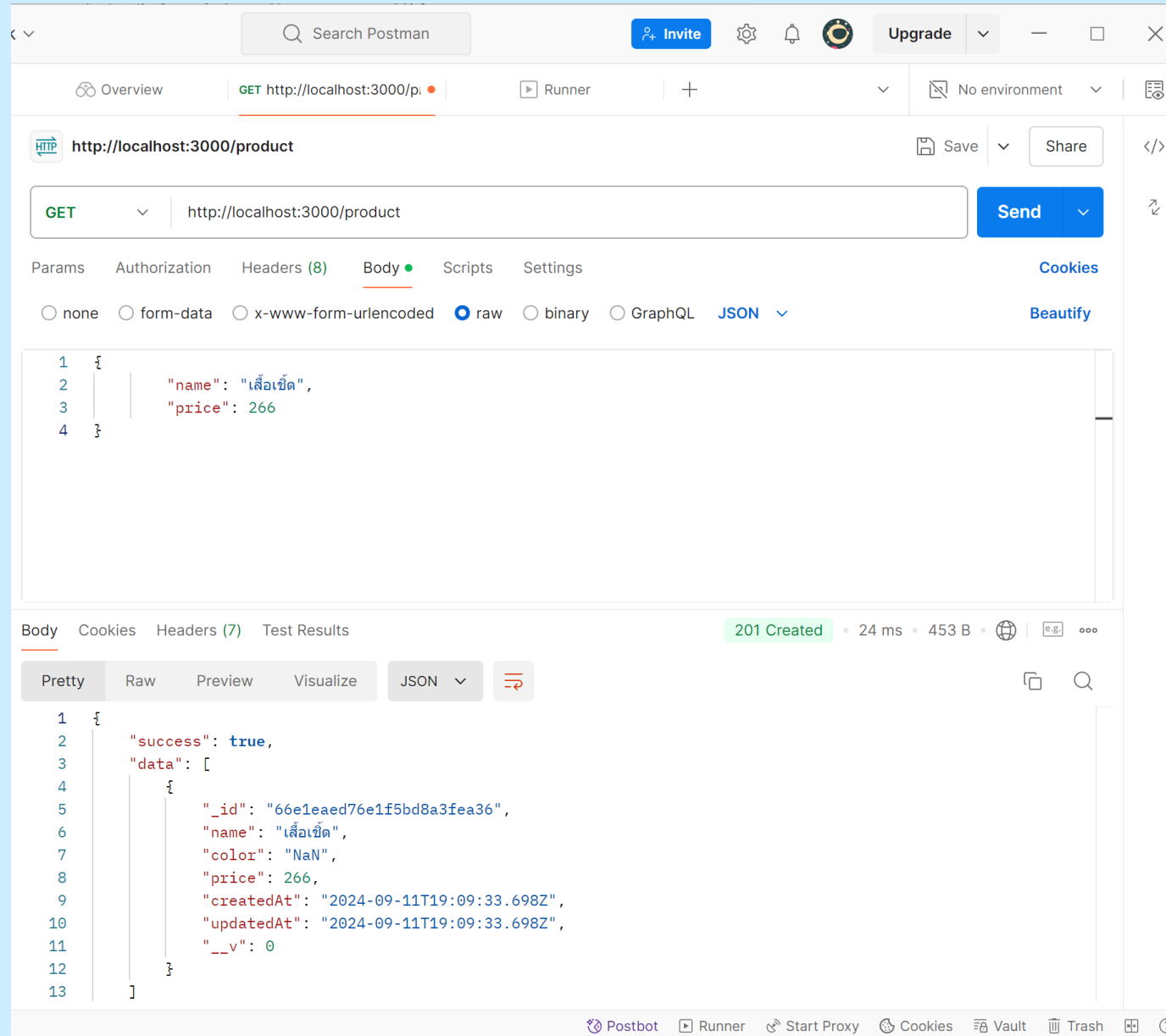
getAll: (query = {}) => { ... }

- เป็นการประกาศ arrow function
- query = {} หมายถึง ถ้าไม่ส่งค่า query มา → ใช้ค่าเริ่มต้นเป็น {} (object ว่าง)
 - {} ใน MongoDB หมายถึง “ค้นหาทุก document”

ProductModel.find(query)

- ProductModel = Mongoose model ที่อ้างอิง collection ใน MongoDB (เช่น products)
- .find(query) = ค้นหาข้อมูลใน collection ตามเงื่อนไขที่ใส่ใน query
 - ถ้า query = {} → จะคืน ข้อมูลทั้งหมด
 - ถ้า query = { color: "red" } → จะคืน document ที่ field color = "red"
- .find เป็น เมธอดของ MongoDB (มีทั้งใน MongoDB driver และ Mongoose) ใช้สำหรับ ค้นหา document (ข้อมูล) จาก collection ตามเงื่อนไขที่กำหนด

ทดลองใช้ postman ส่ง request โดย GET จะต้องมีการแสดงผลกลับมา



ถ้าต้องการดึงข้อมูล product ด้วย id

- ไปที่ ไฟล์ product.service.js เพิ่มโค้ด

```
,findOneById : (id = {})=>{  
  return ProductModel.findOne({_id:id})  
}
```

- ไปที่ ไฟล์ product.controller.js เพิ่มโค้ด

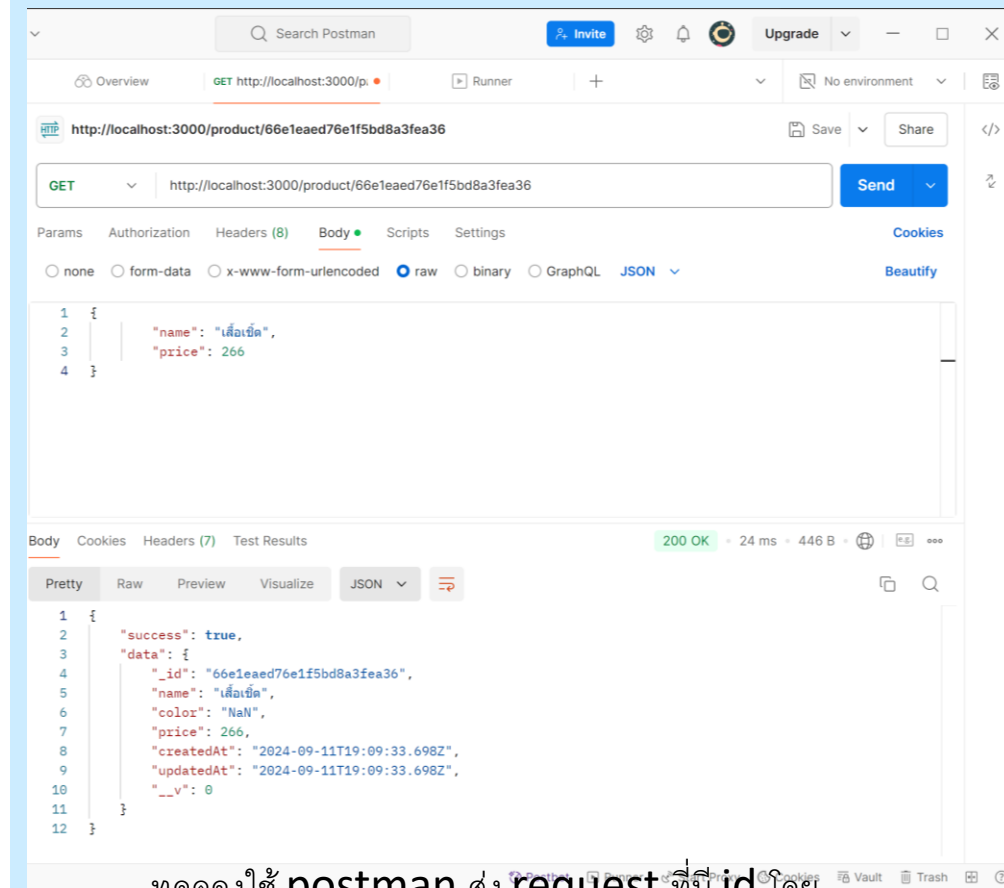
```
,getProductById : async (req,res)=>{  
  const {id} = req.params  
  const products = await ProductService.findOneById(id)  
  res.status(200).json({  
    success:true,  
    data: products  
  })  
}
```

- ไปที่ ไฟล์ product.route.js เพิ่มโค้ด

```
ProductRouter .get('/:id', ProductController.getProductById)
```

findOne คืออะไร

- เป็น เมธอดของ MongoDB/Mongoose
- ใช้ค้นหา document เดียว (เพียง 1 record) จาก collection ที่ตรงกับเงื่อนไข query ที่กำหนด
- ถ้าเจอหลายตัว → จะคืน ตัวแรกที่เจอ ตามลำดับที่เก็บใน database
- ถ้าไม่เจอ → คืนค่า null



ทดลองใช้ postman ส่ง request ที่มี id โดย

GET จะต้องมีข้อมูลแสดงกลับมา

ความแตกต่างระหว่าง PUT และ PATCH

- PUT

- ใช้เพื่อ แทนที่ข้อมูลทั้งหมดของ resource
- ข้อมูลใหม่ที่ส่งมา จะไปแทนที่ทั้งหมด
- ถ้ามี field ไหน ไม่ได้ส่งมา → field นั้นอาจถูก ลบหรือรีเซ็ตเป็นค่า default/null (ขึ้นกับการเขียนโค้ดฝั่ง server)

- PATCH

- ใช้เพื่อ อัปเดตบางส่วนของ resource
- ข้อมูลที่ไม่ได้ส่งมา จะ คงค่าเดิม
- เหมาะกับการแก้ไข field บางตัว ไม่กระทบข้อมูลที่เหลือ

- ไปที่ ไฟล์ product.service.js เพิ่มโค้ด

```
,
  updateOneById : (id,payload)=>{
    return ProductModel.updateOne({_id:id},{ $set:payload})
  }
```

- ไปที่ ไฟล์ product.controller.js เพิ่มโค้ด

```
,
updateProduct : async (req,res)=>{
  const {id} = req.params
  const {name,color,price} = req.body
  const updated = await ProductService.updateOneById(id,{name,color,price})
  res.status(200).json({
    success:true,
    data: updated
  })
}
```

- สร้างไฟล์ update-product.dto.js ในโฟลเดอร์ dto

```
import Joi from 'joi'
export const UpdateProductDto = Joi.object({
  name: Joi.string().optional(),
  color: Joi.string().optional(),
  price: Joi.number().optional()
}))
```

ไม่ได้ระบุ id ใน schema เพราะ id ควรจะอยู่ใน URL params

- ไปที่ ไฟล์ product.route.js เพิ่มโค้ด

```
import { UpdateProductDto } from './update-product.dto.js'
ProductRouter .patch('/:id',validator.body(UpdateProductDto ), ProductController.updateProduct)
```

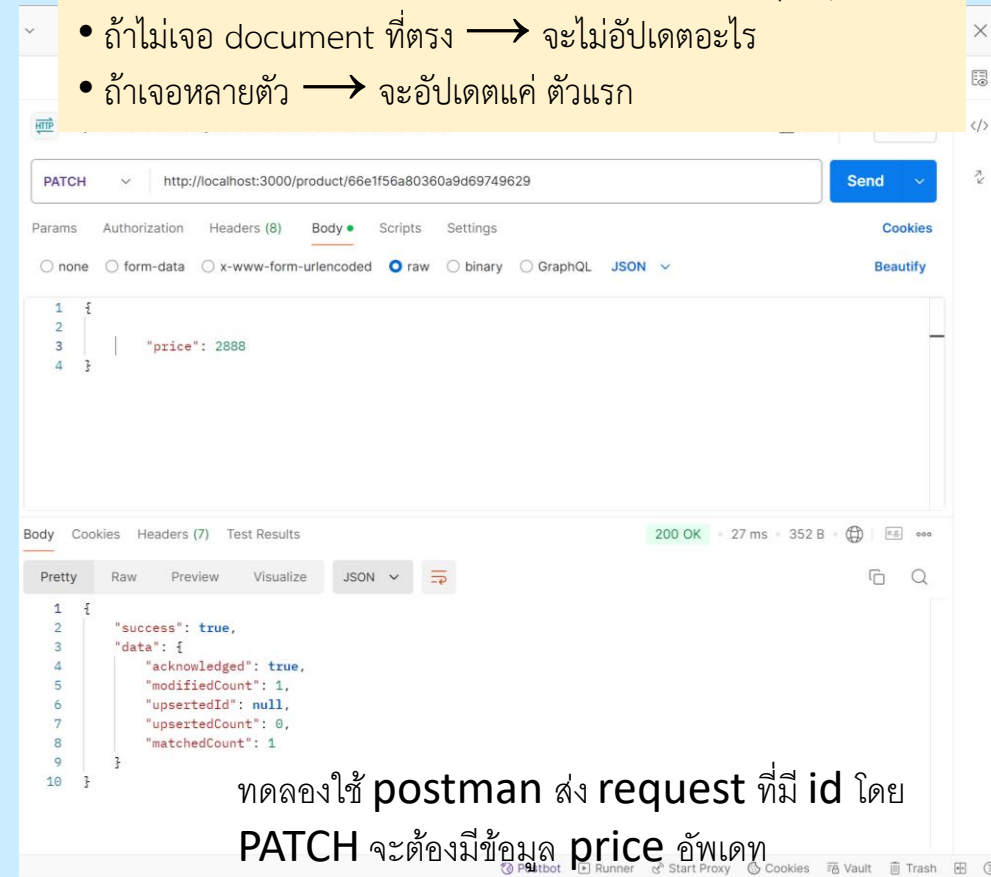
ใช้ patch ในการอัปเดตข้อมูลบางส่วน

ใช้ put ในการอัปเดตข้อมูลทั้งหมดให้ตรงตามข้อมูลใหม่ ข้อมูลที่ไม่ระบุในคำขอ PUT จะถูกแทนที่หรือถูกลบ

ถ้าต้องการอัปเดตข้อมูล product

.updateOne คืออะไร

- เป็น เมธอดของ MongoDB/Mongoose
- ใช้สำหรับ อัปเดต document แรกที่ตรงกับเงื่อนไข (query)
- ถ้าไม่เจอ document ที่ตรง → จะไม่อัปเดตอะไร
- ถ้าเจอหลายตัว → จะอัปเดตแค่ ตัวแรก



ทดลองใช้ postman ส่ง request ที่มี id โดย PATCH จะต้องมีข้อมูล price อัปเดต

ถ้าต้องการลบข้อมูล product

- ไปที่ ไฟล์ product.service.js เพิ่มโค้ด

```
, deleteOneById : (id) =>{  
    return ProductModel.deleteOne({_id:id})  
}
```

- ไปที่ ไฟล์ product.controller.js เพิ่มโค้ด

```
,deleteProduct : async (req,res)=>{  
    const {id} = req.params  
    try {  
        const result = await ProductService.deleteOneById(id)  
        if (result.deletedCount === 0) {  
            return res.status(404).json({ success: false, message: 'Product  
not found' });  
        }  
        res.status(200).json({ success: true, message: 'Product deleted  
successfully' });  
    } catch (error) {  
        res.status(500).json({ success: false, message: error.message });  
    }  
}
```

- ไปที่ ไฟล์ product.route.js เพิ่มโค้ด

```
ProductRouter .delete('/:id', ProductController.deleteProduct)
```

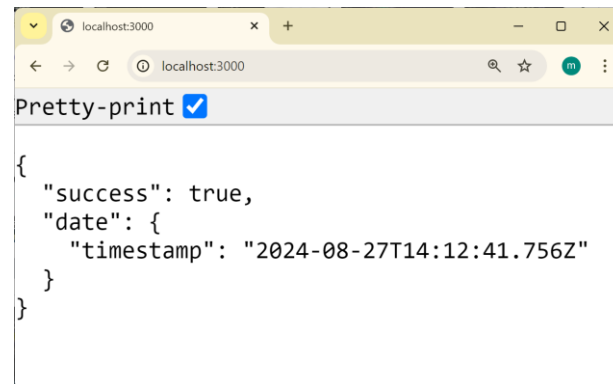
.deleteOne คืออะไร

- เป็น เมธอดของ MongoDB/Mongoose
- ใช้สำหรับ ลบ document ตัวแรกที่ตรงกับเงื่อนไข (filter)
- ถ้าไม่เจอ document → จะไม่ลบอะไร และคืนค่า deletedCount = 0
- deletedCount = 1 → มี document ถูกลบ 1 ตัว
- deletedCount = 0 → ไม่เจอ document ที่ตรงเงื่อนไข

The screenshot shows a web browser interface for a REST client. The URL bar shows `http://localhost:3000/product/66e1f56a80360a9d69749629`. The method dropdown is set to `DELETE`. The response body is displayed in JSON format: `{ "success": true, "message": "Product deleted successfully" }`.

ใบงาน

- 1. สร้างไฟล์ index.js เก็บไว้ที่โฟลเดอร์ mongoDB-รหัสนักศึกษา เพื่อจำลองสร้าง Web Server เมื่อมีการใช้คำสั่ง `npx nodemon index.js` ในการรันจะแสดงผลลัพธ์เป็นข้อความว่า “Server is running on port 3000” (3000 คือการตั้งค่าของ PORT ที่กำหนดในโค้ด) มีการกำหนด status-code เป็น 200 และเมื่อใส่ URL : `http://localhost:3000` จะแสดงรูป



ใบงาน

2. สร้างไฟล์ .env เก็บไว้ที่โฟลเดอร์ mongoDB-รหัสนักศึกษา เพื่อเก็บตัวแปร environment variables ชื่อ MONGODB_CONNECTION ที่เก็บ URL เชื่อมต่อ MongoDB
3. สร้างไฟล์ mongoose.db.js เก็บไว้ที่โฟลเดอร์ src เพื่อใช้โหลด environment variables จากไฟล์ .env และเชื่อมต่อกับ MongoDB
4. สร้างไฟล์ index.config.js เก็บไว้ที่โฟลเดอร์ src จัดการการตั้งค่าการเชื่อมต่อฐานข้อมูลและการตั้งค่าอื่น ๆ สำหรับแอปพลิเคชัน
5. สร้างไฟล์ index.route.js ไว้ที่ src โดย index.route.js เป็นไฟล์ที่ใช้เส้นทางที่เริ่มต้นให้ข้อมูลหนังสือ เริ่มต้นที่ /book

ใบงาน

6. สร้างไฟล์ `book.schema.js` ไว้ที่โฟลเดอร์ `src` เพื่อสร้างโมเดลและกำหนดโครงสร้างของเอกสาร (document) ที่จะจัดเก็บในคอลเล็กชัน `books` ในฐานข้อมูล MongoDB
7. สร้างไฟล์ `book.service.js` ไว้ที่โฟลเดอร์ `src` เพื่อสร้างฟังก์ชันที่ใช้กำหนดการกระทำกับฐานข้อมูล
8. สร้างไฟล์ `create-book.dto.js` และ `update-book.dto.js` โดยใช้ `Joi` สำหรับการตรวจสอบความถูกต้องของข้อมูลเพื่อให้แน่ใจว่าข้อมูลเป็นไปตามรูปแบบที่กำหนดใน `schema` ก่อนการสร้างและอัปเดตข้อมูล
9. สร้างไฟล์ `book.controller.js` ไว้ที่โฟลเดอร์ `src` เพื่อกำหนดฟังก์ชันที่ใช้รองรับการทำงานของ `route` ในไฟล์ `book.route.js`
10. สร้างไฟล์ `book.route.js` ไว้ที่ `src` โดย `book.route.js` เขียนโค้ดเพื่อสร้าง `router` เพื่อ ดึง อัปเดต เพิ่ม ลบข้อมูลหนังสือ โดยเรียกใช้ฟังก์ชันที่อยู่ในไฟล์ `book.controller.js` มาใช้ และใช้ `validator` เป็น `middleware` ใน `route` โดยใช้ไฟล์ในโฟลเดอร์ `dto` ซึ่งคือ `create-book.dto.js` และ `update-book.dto.js`

ใบงาน

- 11. โดยภายใน book.route.js มีการกำหนด router ดังนี้
 - เพิ่มข้อมูลหนังสือ โดยรับข้อมูล title, author จาก request แต่จะกำหนด id ในโค้ด แล้ว response ข้อมูลที่สร้างด้วย status-code เป็น 201 ด้วยเมธอด POST ผ่าน URL: <http://localhost:3000/book> ให้เพิ่มข้อมูลโดยใช้ postman โดยกำหนดข้อมูลดังนี้

id	title	author
1	The Hunger Games	Harper Lee'
2	The Da Vinci Code	Dan Brown
3	The Lord of the Rings: The Fellowship of the Ring	J.R.R. Tolkien

ใบงาน

- 11. โดยภายใน book.route.js มีการกำหนด router ดังนี้ (ต่อ)
 - ดึงข้อมูลรายการหนังสือทุกรายการ ด้วยเมธอด GET ผ่าน URL: `http://localhost:3000/ book`
 - ดึงข้อมูลหนังสือตาม id ที่กำหนดที่ Parameter ใน URL ด้วยเมธอด GET ตัวอย่าง URL คือ `http://localhost:3000/book/1` โดย 1 คือ id ของหนังสือ
 - แก้ไขข้อมูลหนังสือตาม id ที่กำหนดที่ Parameter ใน URL ด้วยเมธอด PATCH ตัวอย่าง URL : `http://localhost:3000/book/3` โดย 3 คือ id ของหนังสือ แล้ว response ข้อมูลที่แก้ไขด้วย status-code เป็น 200
 - ลบข้อมูลหนังสือ ตาม id ที่กำหนดที่ Parameter ใน URL ด้วยเมธอด delete ตัวอย่าง URL : `http://localhost:3000/book/3` โดย 3 คือ id ของหนังสือ แล้ว response ด้วย status-code เป็น 204